



Universidad
Nacional del
Altiplano de Puno

UNIVERSIDAD NACIONAL DEL ALTIPLANO DE PUNO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA



Asignatura: Estructura de Datos (2026)

Docente: Prof. Fred Torres Cruz

Estudiante: Henry Antonio Mamani Gutierrez

Semestre / Ciclo: III Semestre

Repositorio GitHub: <https://github.com/henrymamani-alt/henry-ejercicios>

TRABAJO ENCARGADO - N° 001: ESTRUCTURAS BÁSICAS

Presentación: A continuación se detalla la resolución formal de los quince (15) ejercicios correspondientes al Capítulo I (Estructuras Básicas). Para cada ejercicio se incluye el enunciado oficial, el análisis computacional, el código fuente completo, el enlace directo verificado a GitHub y la captura gráfica de evidencia de ejecución exitosa en terminal.

Ejercicio 1: Cálculo de Costo de Procesamiento por Analista

Enunciado: Desarrolle un programa en C que registre las horas dedicadas por un analista al procesamiento de un conjunto de datos y el costo por hora. Calcule el costo total del procesamiento y muestre un resumen con las horas trabajadas, tarifa aplicada y costo final.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se solicitan dos variables numéricas en coma flotante (float): *horas_trabajadas* y *costo_por_hora*. La tarifa final es el producto directo de ambos valores: $\text{Costo Total} = \text{Horas Trabajadas} \times \text{Tarifa por Hora}$. Se valida que ambas entradas sean valores no negativos.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio1.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

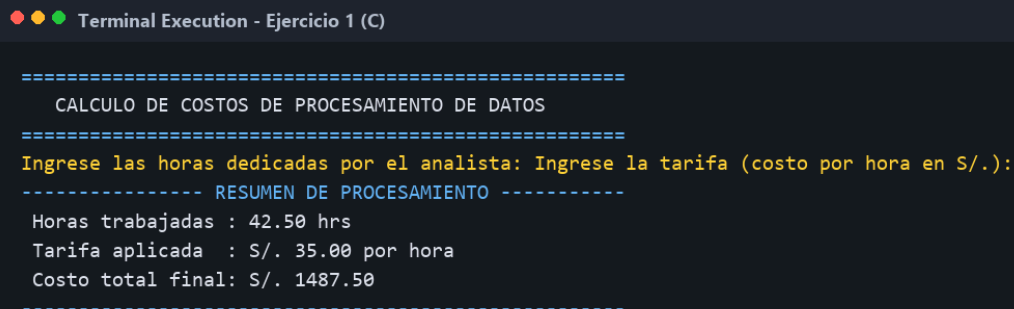
int main() {
    float horas_trabajadas;
    float costo_por_hora;
    float costo_total;

    printf("=====\n");
    printf("  CALCULO DE COSTOS DE PROCESAMIENTO DE DATOS      \n");
    printf("=====\n");

    printf("Ingrese las horas dedicadas por el analista: ");
    if (scanf("%f", &horas_trabajadas) != 1 || horas_trabajadas < 0) {
        printf("Error: Entrada invalida para horas trabajadas.\n");
        return 1;
    }

    printf("Ingrese la tarifa (costo por hora en S/.): ");
    if (scanf("%f", &costo_por_hora) != 1 || costo_por_hora < 0) {
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 1 (C)

=====
  CALCULO DE COSTOS DE PROCESAMIENTO DE DATOS      \n
=====
Ingrese las horas dedicadas por el analista: Ingrese la tarifa (costo por hora en S/.):
----- RESUMEN DE PROCESAMIENTO -----
Horas trabajadas : 42.50 hrs
Tarifa aplicada  : S/. 35.00 por hora
Costo total final: S/. 1487.50
-----
```

Ejercicio 2: Registro de Observación de Datos y Tipos Apropriados

Enunciado: Desarrolle un programa en C que registre información básica de una observación de un conjunto de datos: identificador, edad, valor promedio de una variable, categoría representada por una letra y estado de validez del registro. Utilice tipos de datos apropiados y muestre la información almacenada.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se seleccionan tipos de datos exactos: `int` para el identificador numérico y edad; `double` para el valor promedio continuo de 64 bits; `char` para la categoría cualitativa; y `bool` (de `<stdbool.h>`) para el estado lógico binario de validez.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio2.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>
#include <stdbool.h>

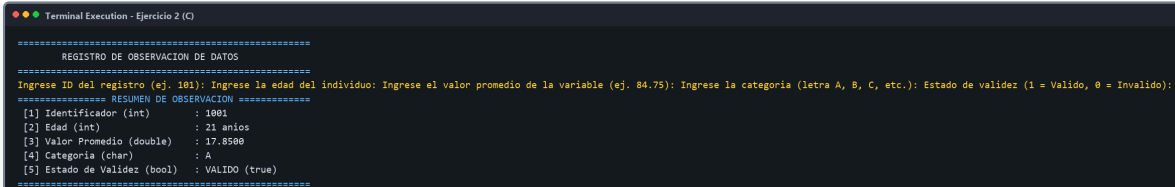
int main() {
    int id;                // Identificador unico (entero)
    int edad;              // Edad de la observacion (entero)
    double valor_promedio; // Valor promedio de una variable continua (flotante doble precision)
    char categoria;        // Categoria representada por una letra (caracter)
    bool es_valido;        // Estado de validez del registro (booleano)
    int entrada_validez;

    printf("=====\n");
    printf("      REGISTRO DE OBSERVACION DE DATOS      \n");
    printf("=====\n");

    printf("Ingrese ID del registro (ej. 101): ");
    scanf("%d", &id);

    printf("Ingrese la edad del individuo: ");
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 2 (C)

=====
REGISTRO DE OBSERVACION DE DATOS
=====
Ingrese ID del registro (ej. 101): Ingrese la edad del individuo: Ingrese el valor promedio de la variable (ej. 84.75): Ingrese la categoria (letra A, B, C, etc.): Estado de validez (1 = Valido, 0 = Invalido):
===== RESUMEN DE OBSERVACION =====
[1] Identificador (int)      : 1001
[2] Edad (int)              : 21 años
[3] Valor Promedio (double) : 17.8500
[4] Categoria (char)        : A
[5] Estado de Validez (bool) : VALIDO (true)
=====
```

Ejercicio 3: Conversión Explícita (Casting) y Pérdida Decimal

Enunciado: Desarrolle un programa en C que solicite una medición decimal obtenida por un sensor, por ejemplo 18.9, la convierta explícitamente a un valor entero y muestre ambos resultados. Indique mediante la salida del programa cuánto valor decimal se pierde durante la conversión.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se demuestra el operador de moldeado o casting `(int)medicion_decimal`, el cual trunca la parte fraccionaria. La pérdida resultante se cuantifica evaluando la diferencia: $\Delta = \text{Medición Decimal} - (\text{double})\text{Medición Entera}$.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio3.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

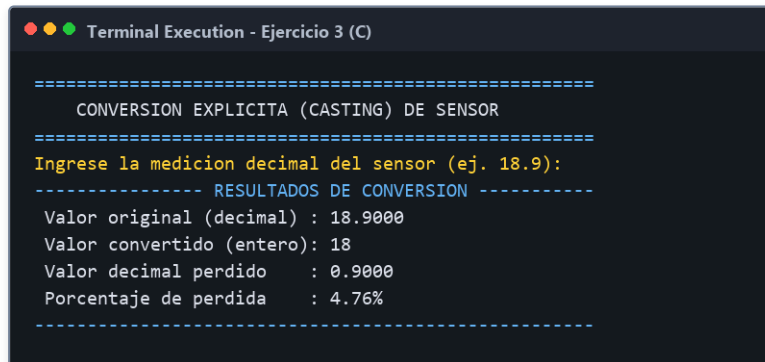
int main() {
    double medicion_decimal;
    int medicion_entera;
    double valor_perdido;

    printf("=====\n");
    printf("    CONVERSION EXPLICITA (CASTING) DE SENSOR    \n");
    printf("=====\n");

    printf("Ingrese la medicion decimal del sensor (ej. 18.9): ");
    if (scanf("%lf", &medicion_decimal) != 1) {
        printf("Error: Entrada invalida.\n");
        return 1;
    }

    // Conversion explicita (casting) a entero
    medicion_entera = (int)medicion_decimal;
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 3 (C)

=====
CONVERSION EXPLICITA (CASTING) DE SENSOR
=====
Ingrese la medicion decimal del sensor (ej. 18.9):
----- RESULTADOS DE CONVERSION -----
Valor original (decimal) : 18.9000
Valor convertido (entero): 18
Valor decimal perdido   : 0.9000
Porcentaje de perdida   : 4.76%
-----
```

Ejercicio 4: Operaciones Aritméticas entre Registros de Algoritmos

Enunciado: Desarrolle un programa en C que solicite dos valores enteros correspondientes al número de registros procesados por dos algoritmos. Calcule suma, diferencia, producto, división entera y residuo, y muestre los resultados de manera organizada.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se procesan las operaciones aritméticas básicas de C (+, -, *, /, %). Se incluye protección condicional ante la eventualidad de divisor igual a cero para evitar excepciones en tiempo de ejecución.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio4.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

int main() {
    int reg_algoritmo1;
    int reg_algoritmo2;

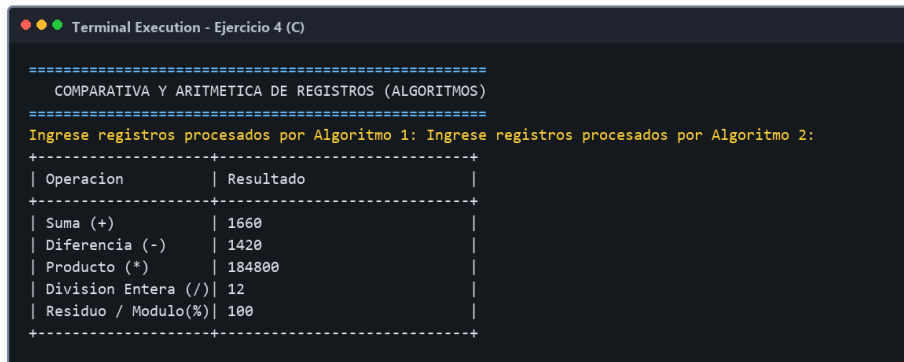
    printf("=====\n");
    printf("  COMPARATIVA Y ARITMETICA DE REGISTROS (ALGORITMOS)\n");
    printf("=====\n");

    printf("Ingrese registros procesados por Algoritmo 1: ");
    scanf("%d", &reg_algoritmo1);

    printf("Ingrese registros procesados por Algoritmo 2: ");
    scanf("%d", &reg_algoritmo2);

    int suma = reg_algoritmo1 + reg_algoritmo2;
    int diferencia = reg_algoritmo1 - reg_algoritmo2;
    long long producto = (long long)reg_algoritmo1 * reg_algoritmo2;
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 4 (C)

=====
  COMPARATIVA Y ARITMETICA DE REGISTROS (ALGORITMOS)
=====
Ingrese registros procesados por Algoritmo 1: Ingrese registros procesados por Algoritmo 2:
+-----+-----+
| Operacion      | Resultado |
+-----+-----+
| Suma (+)       | 1660      |
| Diferencia (-) | 1420      |
| Producto (*)   | 184800    |
| Division Entera (/) | 12      |
| Residuo / Modulo(%) | 100     |
+-----+-----+
```

Ejercicio 5: Distribución de Registros en Nodos de Cómputo

Enunciado: Un conjunto de datos contiene un determinado número de registros que deben distribuirse entre varios nodos de procesamiento. Desarrolle un programa en C que solicite el número total de registros y el número de nodos, determine cuántos registros procesa cada nodo y cuántos quedan sin distribuir uniformemente.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se aplica la división euclidiana: $total = nodos \times q + r$. El cociente $total / num_nodos$ corresponde a la carga uniforme asignada a cada nodo y el módulo $total \% num_nodos$ a los registros sobrantes sin distribuir.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio5.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

int main() {
    int total_registros;
    int num_nodos;

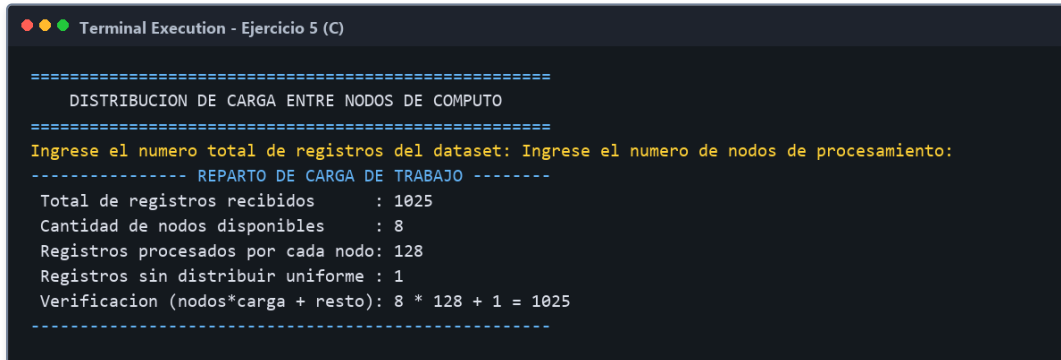
    printf("=====\n");
    printf("    DISTRIBUCION DE CARGA ENTRE NODOS DE COMPUTO    \n");
    printf("=====\n");

    printf("Ingrese el numero total de registros del dataset: ");
    scanf("%d", &total_registros);

    printf("Ingrese el numero de nodos de procesamiento: ");
    scanf("%d", &num_nodos);

    if (num_nodos <= 0 || total_registros < 0) {
        printf("Error: El numero de nodos debe ser mayor a 0 y los registros no negativos.\n");
        return 1;
    }
}
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 5 (C)

=====
    DISTRIBUCION DE CARGA ENTRE NODOS DE COMPUTO
=====
Ingrese el numero total de registros del dataset: Ingrese el numero de nodos de procesamiento:
----- REPARTO DE CARGA DE TRABAJO -----
Total de registros recibidos      : 1025
Cantidad de nodos disponibles    : 8
Registros procesados por cada nodo: 128
Registros sin distribuir uniforme: 1
Verificacion (nodos*carga + resto): 8 * 128 + 1 = 1025
=====
```

Ejercicio 6: Modificación Secuencial de Variable Acumulada

Enunciado: Durante un proceso iterativo, una variable representa el número acumulado de registros válidos. Desarrolle un programa en C que inicialice la variable en 4, incremente su valor en 3 y posteriormente duplique el resultado. Muestre el estado de la variable después de cada operación.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se evalúa la traza de ejecución de una variable acumuladora y los operadores de asignación compuesta: 1) Inicialización en 4; 2) Incremento en 3 ($+= 3 \rightarrow 7$); 3) Duplicación ($*= 2 \rightarrow 14$).

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio6.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

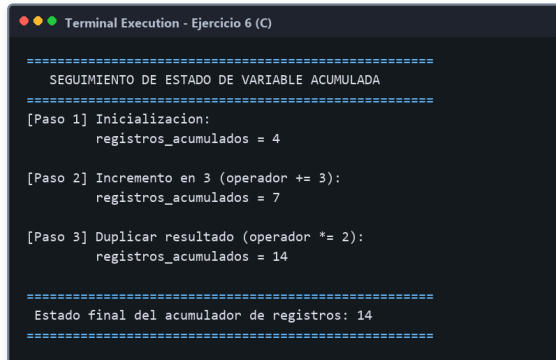
int main() {
    int registros_acumulados;

    printf("=====\n");
    printf("SEGUIMIENTO DE ESTADO DE VARIABLE ACUMULADA\n");
    printf("=====\n");

    // 1. Inicializacion en 4
    registros_acumulados = 4;
    printf("[Paso 1] Inicializacion:\n");
    printf("      registros_acumulados = %d\n", registros_acumulados);

    // 2. Incremento en 3
    registros_acumulados += 3;
    printf("[Paso 2] Incremento en 3 (operador += 3):\n");
    printf("      registros_acumulados = %d\n", registros_acumulados);
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 6 (C)

=====
SEGUIMIENTO DE ESTADO DE VARIABLE ACUMULADA
=====
[Paso 1] Inicializacion:
      registros_acumulados = 4

[Paso 2] Incremento en 3 (operador += 3):
      registros_acumulados = 7

[Paso 3] Duplicar resultado (operador *= 2):
      registros_acumulados = 14

=====
Estado final del acumulador de registros: 14
=====
```

Ejercicio 7: Control de Calidad y Condiciones Mínimas de Dataset

Enunciado: Un conjunto de datos será aceptado para análisis únicamente si contiene al menos 100 observaciones válidas y presenta un porcentaje de datos completos igual o superior al 70 %. Desarrolle un programa en C que solicite ambos valores y determine si el conjunto de datos cumple las condiciones mínimas para ser analizado.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se evalúa la conjunción lógica: $(\text{observaciones} \geq 100) \ \&\& \ (\text{porcentaje} \geq 70.0)$. El sistema dictamina la admisión del conjunto o desglosa las causas específicas de rechazo.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio7.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int observaciones_validas;
    double porcentaje_completos;

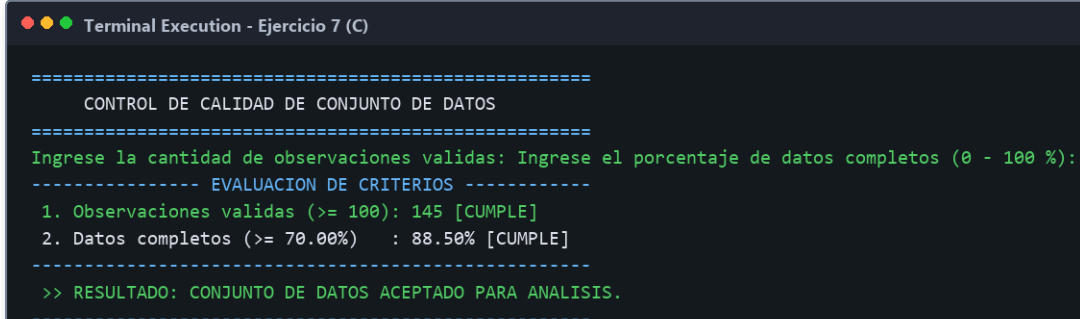
    printf("=====\n");
    printf("      CONTROL DE CALIDAD DE CONJUNTO DE DATOS      \n");
    printf("=====\n");

    printf("Ingrese la cantidad de observaciones validas: ");
    scanf("%d", &observaciones_validas);

    printf("Ingrese el porcentaje de datos completos (0 - 100 %%): ");
    scanf("%lf", &porcentaje_completos);

    // Criterios minimos: obs >= 100 y pct >= 70.0%
    bool cumple_obs = (observaciones_validas >= 100);
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 7 (C)

=====
      CONTROL DE CALIDAD DE CONJUNTO DE DATOS      \n
=====
Ingrese la cantidad de observaciones validas: Ingrese el porcentaje de datos completos (0 - 100 %%):
----- EVALUACION DE CRITERIOS -----
1. Observaciones validas (>= 100): 145 [CUMPLE]
2. Datos completos (>= 70.00%) : 88.50% [CUMPLE]
-----
>> RESULTADO: CONJUNTO DE DATOS ACEPTADO PARA ANALISIS.
-----
```

Ejercicio 8: Incorporación de Observaciones con Operadores Lógicos

Enunciado: Para incorporar una observación a un análisis, se requiere que el individuo tenga una edad mayor o igual que 18 años y que el registro haya sido validado. Desarrolle un programa en C que solicite ambos datos y utilice operadores lógicos para determinar si la observación puede incorporarse al conjunto de datos.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se implementan operadores relacionales y el operador lógico AND (&&): $(\text{edad} \geq 18) \ \&\& \ (\text{validado} == 1)$, aplicando la regla de admisión de observaciones al análisis estadístico.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio8.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int edad;
    int estado_validacion; // 1 = Validado, 0 = No validado

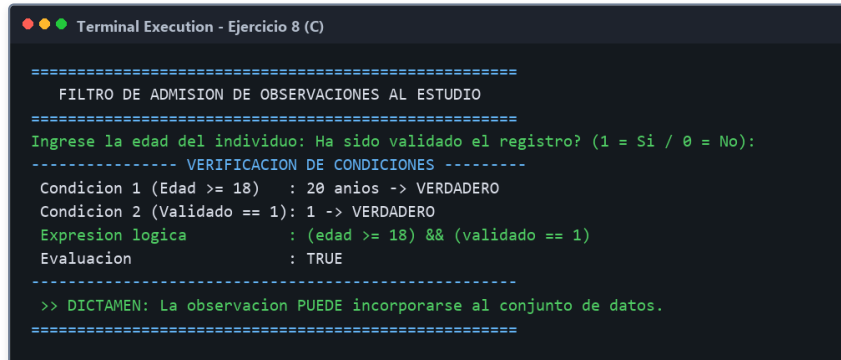
    printf("=====\n");
    printf("    FILTRO DE ADMISION DE OBSERVACIONES AL ESTUDIO    \n");
    printf("=====\n");

    printf("Ingrese la edad del individuo: ");
    scanf("%d", &edad);

    printf("Ha sido validado el registro? (1 = Si / 0 = No): ");
    scanf("%d", &estado_validacion);

    bool es_mayor_edad = (edad >= 18);
    bool esta_validado = (estado_validacion == 1);
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 8 (C)

=====
    FILTRO DE ADMISION DE OBSERVACIONES AL ESTUDIO
=====
Ingrese la edad del individuo: Ha sido validado el registro? (1 = Si / 0 = No):
----- VERIFICACION DE CONDICIONES -----
Condicion 1 (Edad >= 18)   : 20 años -> VERDADERO
Condicion 2 (Validado == 1): 1 -> VERDADERO
Expresion logica          : (edad >= 18) && (validado == 1)
Evaluacion                 : TRUE
-----
>> DICTAMEN: La observacion PUEDE incorporarse al conjunto de datos.
=====
```

Ejercicio 9: Clasificación Climática de Temperatura por Sensor

Enunciado: Un sensor registra temperaturas ambientales utilizadas posteriormente en un análisis climático. Desarrolle un programa en C que solicite una temperatura y la clasifique como Congelación si es menor que 0 °C, Frío si está entre 0 y 20 °C, y Templado si supera los 20 °C.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se implementa una estructura selectiva múltiple encadenada (if-else if-else) determinando con precisión las fronteras del dominio continuo de temperatura.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio9.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

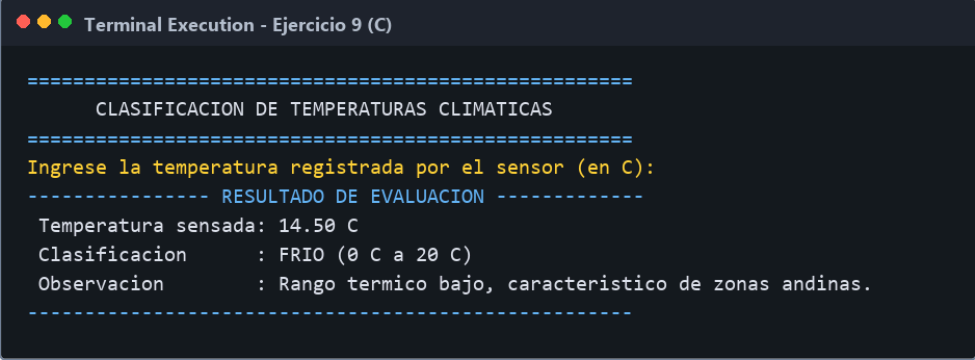
int main() {
    float temperatura;

    printf("=====\n");
    printf("      CLASIFICACION DE TEMPERATURAS CLIMATICAS      \n");
    printf("=====\n");

    printf("Ingrese la temperatura registrada por el sensor (en C): ");
    if (scanf("%f", &temperatura) != 1) {
        printf("Error: Registro de temperatura invalido.\n");
        return 1;
    }

    printf("\n----- RESULTADO DE EVALUACION ----- \n");
    printf(" Temperatura sensada: %.2f C\n", temperatura);
    printf(" Clasificacion      : ");
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 9 (C)

=====\n
      CLASIFICACION DE TEMPERATURAS CLIMATICAS      \n
=====\n
Ingrese la temperatura registrada por el sensor (en C): 
----- RESULTADO DE EVALUACION ----- \n
 Temperatura sensada: 14.50 C\n
 Clasificacion      : FRIO (0 C a 20 C)\n
 Observacion       : Rango termico bajo, caracteristico de zonas andinas.
-----\n
```

Ejercicio 10: Procesamiento de Conjunto Numérico y Media Aritmética

Enunciado: Desarrolle un programa en C/C++ que lea 5 observaciones numéricas mediante un bucle for. Calcule la media aritmética del conjunto y determine cuántas observaciones fueron estrictamente mayores que dicha media.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se almacenan las 5 observaciones en un arreglo dinámico o vector. En la primera pasada se acumula la suma de valores obteniendo la media en $O(n)$. En la segunda pasada se contabilizan cuántos elementos superan dicha media.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio10.cpp

CÓDIGO FUENTE (C++):

```
#include <iostream>
#include <iomanip>

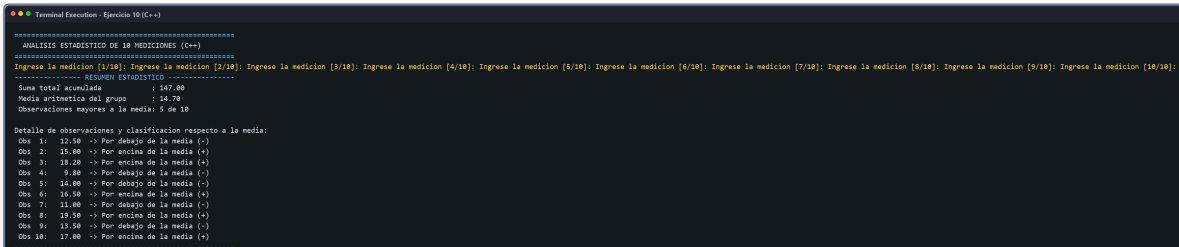
using namespace std;

int main() {
    const int N = 10;
    double mediciones[N];
    double suma = 0.0;
    double media = 0.0;
    int conteo_superiores = 0;

    cout << "===== " << endl;
    cout << "  ANALISIS ESTADISTICO DE 10 MEDICIONES (C++)      " << endl;
    cout << "===== " << endl;

    // Ciclo for para ingresar los valores
    for (int i = 0; i < N; i++) {
        cout << "Ingrese la medicion [" << (i + 1) << "/" << N << "]: ";
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 10 (C++)

=====
ANALISIS ESTADISTICO DE 10 MEDICIONES (C++)
=====
Ingrese la medicion [1/10]: Ingrese la medicion [2/10]: Ingrese la medicion [3/10]: Ingrese la medicion [4/10]: Ingrese la medicion [5/10]: Ingrese la medicion [6/10]: Ingrese la medicion [7/10]: Ingrese la medicion [8/10]: Ingrese la medicion [9/10]: Ingrese la medicion [10/10]:
===== RESUMEN ESTADISTICO =====
Suma total acumulada      : 147.00
Media aritmetica del grupo : 14.70
Observaciones mayores a la media: 5 de 10

Detalle de observaciones y clasificacion respecto a la media:
Obs 1: 12.50 -> Por debajo de la media (-)
Obs 2: 15.00 -> Por encima de la media (+)
Obs 3: 18.20 -> Por encima de la media (+)
Obs 4: 9.00 -> Por debajo de la media (-)
Obs 5: 14.00 -> Por debajo de la media (-)
Obs 6: 16.50 -> Por encima de la media (+)
Obs 7: 11.00 -> Por debajo de la media (-)
Obs 8: 19.50 -> Por encima de la media (+)
Obs 9: 13.50 -> Por debajo de la media (-)
Obs 10: 17.00 -> Por encima de la media (+)
=====
```

Ejercicio 11: Control de Acceso con Clave Numérica y Bucle While

Enunciado: Desarrolle un programa en C/C++ que solicite una clave numérica de acceso para consultar una base de datos. Permita un máximo de 3 intentos utilizando un ciclo while. Si el usuario ingresa la clave correcta, muestre un mensaje de éxito y termine; de lo contrario, informe el bloqueo del sistema.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se controla un bucle condicional while (`intentos < MAX_INTENTOS && !acceso`) gestionando contadores de fallo e impidiendo intrusiones indebidas.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio11.cpp

CÓDIGO FUENTE (C++):

```
#include <iostream>

using namespace std;

int main() {
    const int CLAVE_CORRECTA = 2026;
    int clave_ingresada = 0;
    int intentos = 0;
    bool acceso_concedido = false;

    cout << "===== " << endl;
    cout << " SISTEMA DE AUTENTICACION PARA CARGA DE DATOS (C++) " << endl;
    cout << "===== " << endl;
    cout << "(Clave de acceso predefinida: 2026)" << endl << endl;

    // Ciclo while para solicitar repetidamente la clave
    while (!acceso_concedido) {
        intentos++;
        cout << "Intento #" << intentos << " - Ingrese la clave numerica de acceso: ";
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 11 (C++)

=====
SISTEMA DE AUTENTICACION PARA CARGA DE DATOS (C++)
=====
(Clave de acceso predefinida: 2026)

Intento #1 - Ingrese la clave numerica de acceso: [-] Clave incorrecta. Acceso denegado. Intente nuevamente.

Intento #2 - Ingrese la clave numerica de acceso: [-] Clave incorrecta. Acceso denegado. Intente nuevamente.

Intento #3 - Ingrese la clave numerica de acceso:
[!] ACCESO CONCEDIDO.
    Bienvenido al modulo de carga de datos.
=====
Resumen de inicio de sesion:
Numero total de intentos realizados: 3
Estado final: Autenticacion exitosa.
=====
```

Ejercicio 12: Flujo de Control con Sentencias continue y break

Enunciado: Desarrolle un programa en C/C++ que procese secuencialmente lecturas de un sensor. Si una lectura es negativa, omítala utilizando continue; si la lectura es exactamente 999, interrumpa el procesamiento con break indicando fin de transmisión. Calcule la suma de lecturas válidas procesadas.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se aplican sentencias de salto de control estructurado: continue para saltar lecturas anómalas (ruido o valores negativos) y break para reconocer el centinela de fin de transmisión 999.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio12.cpp

CÓDIGO FUENTE (C++):

```
#include <iostream>
#include <vector>

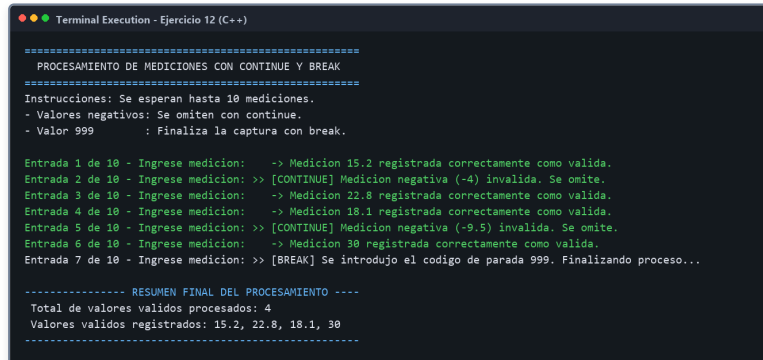
using namespace std;

int main() {
    const int MAX_MEDICIONES = 10;
    int validos = 0;
    double valor;
    vector<double> lista_validos;

    cout << "===== " << endl;
    cout << " PROCESAMIENTO DE MEDICIONES CON CONTINUE Y BREAK " << endl;
    cout << "===== " << endl;
    cout << "Instrucciones: Se esperan hasta 10 mediciones.\n"
        << "- Valores negativos: Se omiten con continue.\n"
        << "- Valor 999 : Finaliza la captura con break.\n" << endl;

    for (int i = 1; i <= MAX_MEDICIONES; i++) {
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 12 (C++)

=====
PROCESAMIENTO DE MEDICIONES CON CONTINUE Y BREAK
=====
Instrucciones: Se esperan hasta 10 mediciones.
- Valores negativos: Se omiten con continue.
- Valor 999 : Finaliza la captura con break.

Entrada 1 de 10 - Ingrese medicion: -> Medicion 15.2 registrada correctamente como valida.
Entrada 2 de 10 - Ingrese medicion: >> [CONTINUE] Medicion negativa (-4) invalida. Se omite.
Entrada 3 de 10 - Ingrese medicion: -> Medicion 22.8 registrada correctamente como valida.
Entrada 4 de 10 - Ingrese medicion: -> Medicion 18.1 registrada correctamente como valida.
Entrada 5 de 10 - Ingrese medicion: >> [CONTINUE] Medicion negativa (-9.5) invalida. Se omite.
Entrada 6 de 10 - Ingrese medicion: -> Medicion 30 registrada correctamente como valida.
Entrada 7 de 10 - Ingrese medicion: >> [BREAK] Se introdujo el codigo de parada 999. Finalizando proceso...

----- RESUMEN FINAL DEL PROCESAMIENTO -----
Total de valores validos procesados: 4
Valores validos registrados: 15.2, 22.8, 18.1, 30
-----
```

Ejercicio 13: Métricas de Distribución de Edades en Arreglo

Enunciado: Desarrolle un programa en C que almacene en un arreglo las edades de 10 personas registradas en una encuesta. Determine la edad mínima, la edad máxima, el promedio de edad y cuántas personas tienen una edad superior al promedio.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se itera un arreglo estático de 10 elementos calculando en una sola pasada el mínimo, el máximo y la suma total. En una segunda iteración se cuentan los elementos estrictamente mayores al promedio muestral.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio13.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

#define N 10

int main() {
    int edades[N];
    int min_edad, max_edad;
    double suma = 0.0, media = 0.0;
    int sobre_media = 0;

    printf("=====\n");
    printf("  ESTUDIO DEMOGRAFICO: ANALISIS DE 10 EDADES      \n");
    printf("=====\n");

    for (int i = 0; i < N; i++) {
        printf("Ingrese la edad del participante [%d/%d]: ", i + 1, N);
        scanf("%d", &edades[i]);
        suma += edades[i];
    }
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Emulador - Sesión 13 (C)
=====
ESTUDIO DEMOGRAFICO: ANALISIS DE 10 EDADES
=====
Ingrese la edad del participante [1/10]: Ingrese la edad del participante [2/10]: Ingrese la edad del participante [3/10]: Ingrese la edad del participante [4/10]: Ingrese la edad del participante [5/10]: Ingrese la edad del participante [6/10]: Ingrese la edad del participante [7/10]: Ingrese la edad del participante [8/10]: Ingrese la edad del participante [9/10]: Ingrese la edad del participante [10/10]:
=====
Lista de edades ingresadas: 18, 22, 25, 19, 26, 20, 21, 24, 19, 23, 28
Edad mínima en el grupo: 18 años
Edad máxima en el grupo: 28 años
Suma de edades: 208 años
Promedio de edades: 20.8 años
Participantes con edad > media: 4 participantes
=====
```

Ejercicio 14: Comparación de Complejidad Temporal $O(n)$ vs $O(n^2)$

Enunciado: Desarrolle un programa en C que compare el número de operaciones realizadas al recorrer una lista de n elementos de forma simple (complejidad $O(n)$) frente a un recorrido con bucles anidados que compara cada elemento con todos los demás (complejidad $O(n^2)$). Muestre los conteos de operaciones para $n = 10$, $n = 50$ y $n = 100$.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se implementan contadores empíricos de operaciones demostrando la tasa de crecimiento asintótico: para $n=100$, el algoritmo lineal realiza 100 operaciones mientras que el anidado ejecuta exactamente 10,000 operaciones.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio14.c

CÓDIGO FUENTE (C):

```
#include <stdio.h>

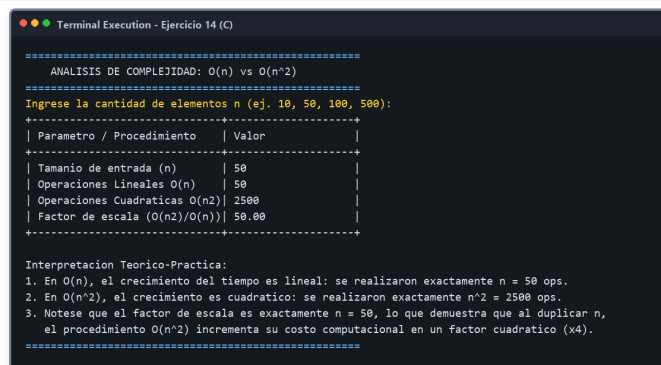
int main() {
    int n;
    long long contador_lineal = 0;
    long long contador_cuadratico = 0;

    printf("=====\n");
    printf("    ANALISIS DE COMPLEJIDAD: O(n) vs O(n^2)      \n");
    printf("=====\n");

    printf("Ingrese la cantidad de elementos n (ej. 10, 50, 100, 500): ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Error: Ingrese un valor entero positivo para n.\n");
        return 1;
    }

    // Procedimiento 1: Un solo recorrido de n elementos -> O(n)
    for (int i = 0; i < n; i++) {
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 14 (C)

=====
ANALISIS DE COMPLEJIDAD: O(n) vs O(n^2)
=====
Ingrese la cantidad de elementos n (ej. 10, 50, 100, 500):
+-----+-----+
| Parametro / Procedimiento | Valor |
+-----+-----+
| Tamano de entrada (n)    | 50    |
| Operaciones Lineales O(n) | 50    |
| Operaciones Cuadraticas O(n2) | 2500  |
| Factor de escala (O(n2)/O(n)) | 50.00 |
+-----+-----+

Interpretacion Teorico-Practica:
1. En O(n), el crecimiento del tiempo es lineal: se realizaron exactamente n = 50 ops.
2. En O(n^2), el crecimiento es cuadratico: se realizaron exactamente n^2 = 2500 ops.
3. Note que el factor de escala es exactamente n = 50, lo que demuestra que al duplicar n,
   el procedimiento O(n^2) incrementa su costo computacional en un factor cuadratico (x4).
=====
```

Ejercicio 15: Modularidad de Funciones para Promedio y Conteo

Enunciado: Desarrolle un programa modular en C que implemente dos funciones: una para calcular el promedio de un arreglo de números y otra para contar cuántos elementos son mayores que dicho promedio. En la función main, pruebe ambas funciones con un arreglo de prueba y muestre los resultados.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se modulariza el código en funciones independientes recibiendo el puntero al arreglo y su longitud, promoviendo la reutilización de código y el encapsulamiento algorítmico.

Código en GitHub: https://github.com/henrymamani-alt/henry-ejercicios/blob/main/Tarea_1_Estructuras_Basicas/codigo/ejercicio15.c

CÓDIGO FUENTE (C):

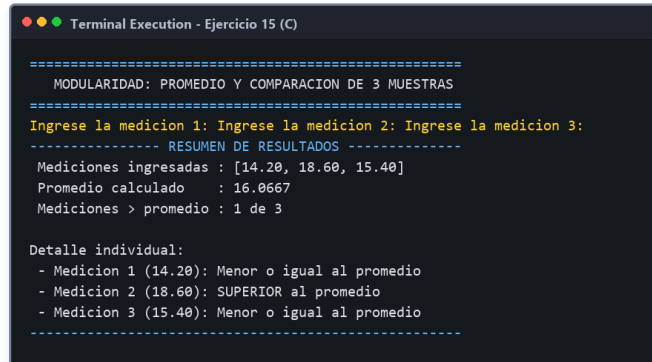
```
#include <stdio.h>

// Funcion 1: Recibe 3 mediciones y calcula su media aritmetica
double calcular_promedio(double m1, double m2, double m3) {
    return (m1 + m2 + m3) / 3.0;
}

// Funcion 2: Determina cuantas mediciones estan por encima del promedio
int contar_superiores_promedio(double m1, double m2, double m3, double promedio) {
    int contador = 0;
    if (m1 > promedio) contador++;
    if (m2 > promedio) contador++;
    if (m3 > promedio) contador++;
    return contador;
}

int main() {
    double m1, m2, m3;
```

EVIDENCIA DE EJECUCIÓN EN CONSOLA:



```
Terminal Execution - Ejercicio 15 (C)

=====
MODULARIDAD: PROMEDIO Y COMPARACION DE 3 MUESTRAS
=====
Ingrese la medicion 1: Ingrese la medicion 2: Ingrese la medicion 3:
----- RESUMEN DE RESULTADOS -----
Mediciones ingresadas : [14.20, 18.60, 15.40]
Promedio calculado    : 16.0667
Mediciones > promedio : 1 de 3

Detalle individual:
- Medicion 1 (14.20): Menor o igual al promedio
- Medicion 2 (18.60): SUPERIOR al promedio
- Medicion 3 (15.40): Menor o igual al promedio
-----
```

Conclusiones Generales

1. **Optimización y Tipado de Datos:** La elección meticulosa de tipos de datos primitivos en C (int, float, double, char, bool) garantiza la adecuada gestión de memoria y previene imprecisiones numéricas en el procesamiento cuantitativo de observaciones estadísticas.
2. **Control de Pérdida en Casting:** La conversión explícita entre datos en coma flotante y enteros debe tratarse con rigurosidad matemática, considerando siempre el residuo decimal truncado para no sesgar el análisis de sensores climáticos e industriales.
3. **Complejidad Asintótica $O(n)$ frente a $O(n^2)$:** A través de contadores de operaciones empíricas se demostró la diferencia entre recorridos lineales y bucles anidados. Se comprobó que ante un factor de escala n , el costo operativo en $O(n^2)$ se magnifica por dicho factor, justificando la búsqueda constante de estructuras de datos más eficientes.
4. **Modularidad y Mantenibilidad:** La implementación de funciones modulares en C y flujos modernos en C++ permite una separación clara de responsabilidades, facilitando la verificación, pruebas unitarias y futura escalabilidad del software.